Coding for AI – Problem Set 7
The Autonomous Payload Packet Parser

Name: Tanishka Hira
Date: 14/09/2026

Advanced Function Signatures

Think of the `/` symbol as a positional boundary—everything before it is positional-only. That means you have to pass `raw_packet` and `delimiter` by their order, not by name.

On the flip side, the `*` symbol acts as a keyword boundary. Any parameter coming after it, like `correction_offset`, is keyword-only, so you'll need to pass it explicitly, for example `correction_offset=value`.

Lastly, using `[:]` creates a fresh slice of the list. By looping over `raw_packet[1][:]` inside the list comprehension, the function constructs a brand-new list of corrected sensors. This protects the original packet from being altered or linked to the new data.

Python Code

```
"""
Coding for AI - Problem Set 7
Project: The Autonomous Payload Packet Parser

Name: Tanishka Hira
Date: 14/09/2026
Advanced Function Signatures

Think of the / symbol as a positional boundary--everything before it
is positional-only. That means you have to pass raw_packet and
delimiter by their order, not by name.

On the flip side, the * symbol acts as a keyword boundary. Any
parameter coming after it, like correction_offset, is keyword-only,
so you'll need to pass it explicitly (e.g.,
correction_offset=value).

Lastly, using [:] creates a fresh slice of the list. By looping over
raw_packet[1][:] inside the list comprehension, the function
constructs a brand-new list of corrected sensors. This protects your
original packet from being altered or linked to the new data.
"""

def parse_payload(raw_packet: list, delimiter: str, /, *,
                  correction_offset: int = 0) -> tuple:
```

```
    status_tokens = raw_packet[2].split(delimiter)
    corrected_sensors = [
        value if "ERROR" in status_tokens
        else value + correction_offset
        for value in raw_packet[1][:]
    ]
    return raw_packet[0], corrected_sensors


# -- TEST 1: NORMAL PACKET WITH CORRECTION --

packet1 = [
    501,
    [12.5, 13.0, 11.8],
    "NOMINAL|CALIBRATED"
]

original_sensors1 = packet1[1][:]

packet_id1, sensors1 = parse_payload(
    packet1,
    "|",
    correction_offset=2
)

print("Test 1 ->", packet_id1, sensors1)

assert packet1[1] == original_sensors1
assert id(sensors1) != id(packet1[1])

print("Test 1 side-effect assertions passed.")


# -- TEST 2: ERROR PACKET --

packet2 = [
    502,
    [9.0, 8.5],
    "ERROR|SENSOR_FAULT"
]

original_sensors2 = packet2[1][:]

packet_id2, sensors2 = parse_payload(
    packet2,
    "|",
    correction_offset=5
```

```
)

print("Test 2 ->", packet_id2, sensors2)

assert sensors2 == [9.0, 8.5]
assert packet2[1] == original_sensors2
assert id(sensors2) != id(packet2[1])

print("Test 2 side-effect assertions passed.")


# -- TEST 3: DEFAULT CORRECTION OFFSET --

packet3 = [
    503,
    [1.0, 2.0],
    "NOMINAL"
]

packet_id3, sensors3 = parse_payload(
    packet3,
    "|"
)

print("Test 3 (default offset) ->", packet_id3, sensors3)


# -- TEST 4: ILLEGAL KEYWORD CALL --

try:
    parse_payload(
        raw_packet=packet1,
        delimiter="|"
    )
except TypeError as error:
    print("Test 4 passed. TypeError raised as expected:", error)


# -- TEST 5: ILLEGAL POSITIONAL CALL --

try:
    parse_payload(
        packet1,
        "|",
        2
    )
except TypeError as error:
```

```
    print("Test 5 passed. TypeError raised as expected:", error)


# -- EDGE CASE 1 --

packet4 = [
    504,
    [5.0, 10.0],
    "NOMINAL|CALIBRATED|LOW_POWER"
]

packet_id4, sensors4 = parse_payload(
    packet4,
    "|",
    correction_offset=2
)

print("Edge Case 1 ->", packet_id4, sensors4)
assert sensors4 == [7.0, 12.0]


# -- EDGE CASE 2 --

packet5 = [
    505,
    [],
    "NOMINAL"
]

packet_id5, sensors5 = parse_payload(
    packet5,
    "|",
    correction_offset=2
)

print("Edge Case 2 ->", packet_id5, sensors5)
assert sensors5 == []


# -- EDGE CASE 3 --

packet6 = [
    506,
    [4.0, 8.0],
    "NOMINAL"
]

```

```
_, default_result = parse_payload(
    packet6,
    "|"
)

_, explicit_zero_result = parse_payload(
    packet6,
    "|",
    correction_offset=0
)

print("Edge Case 3 - Default ->", default_result)
print("Edge Case 3 - Explicit 0 ->", explicit_zero_result)

assert default_result == explicit_zero_result

print("All edge-case tests passed.")
```

Submission Note

These two invalid calls show how `/` and `*` put guardrails on Python function signatures. In the first mistake, I tried passing `raw_packet` and `delimiter` as named arguments. Python threw a `TypeError` right away because anything before `/` has to be passed by position, not name. In the second mistake, I tried passing `correction_offset` as a plain third positional argument. Python snapped back with another `TypeError` because anything after `*` must be named.

To make a valid call, the first two arguments must be positional, like `parse_payload(packet1, "|")`, and any offset value has to be explicitly labeled, like `correction_offset=2`. While these rules feel strict, they make your code much easier to read and stop you from accidentally swapping arguments by mistake.